LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Mohamad Dwi Afrizza (2609106001)

Kelas (A1’26)

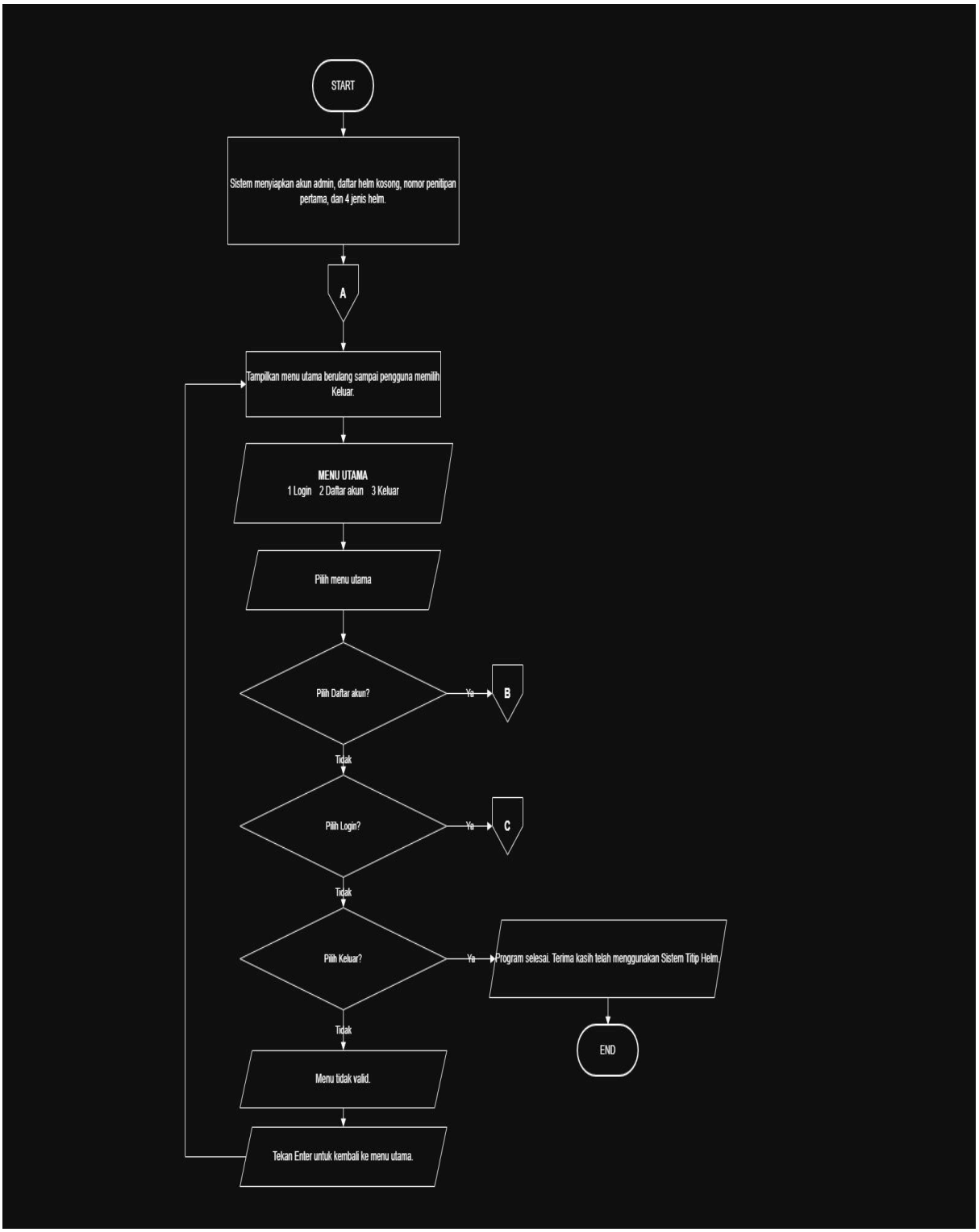
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

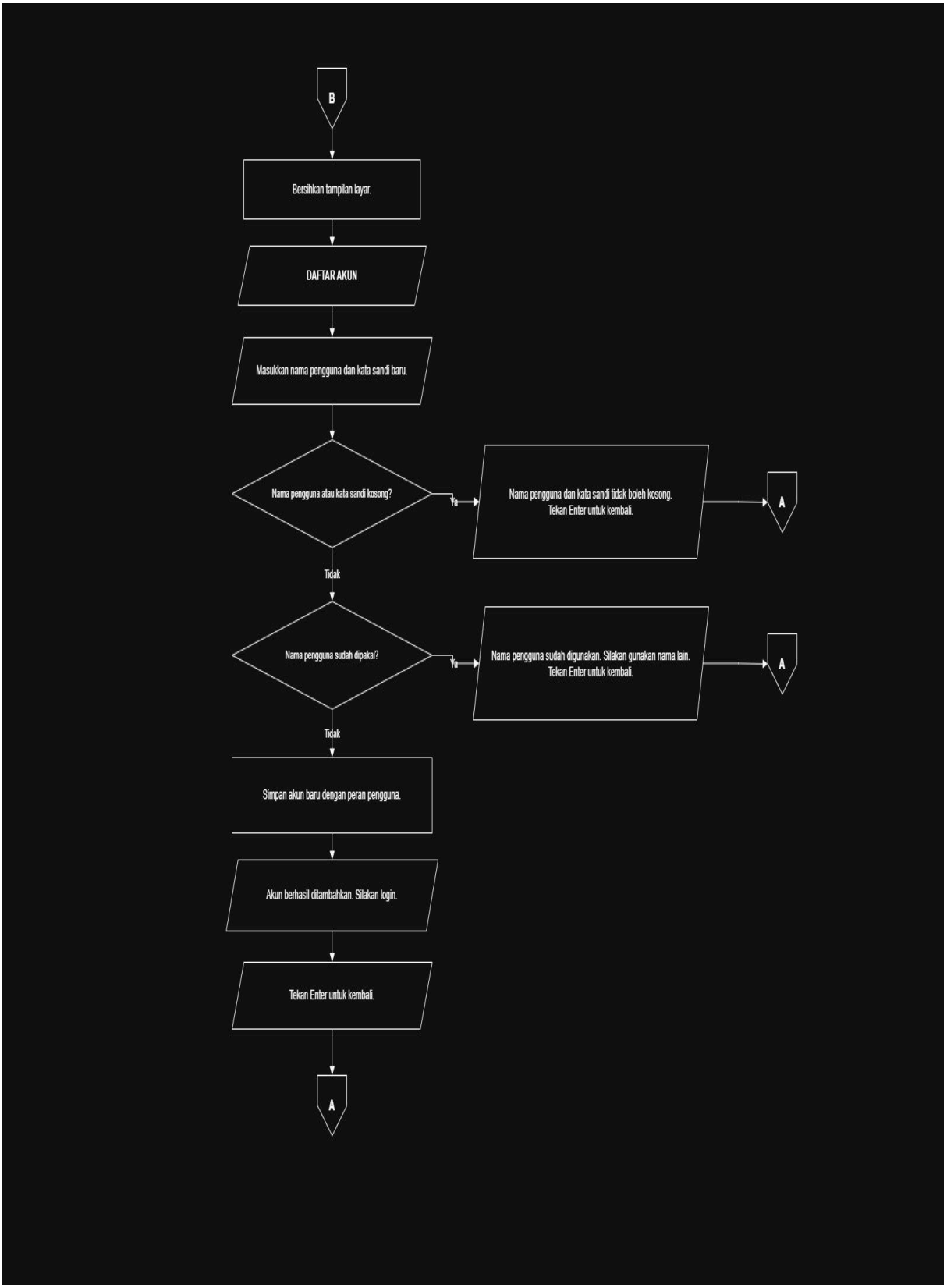
SAMARINDA

2026

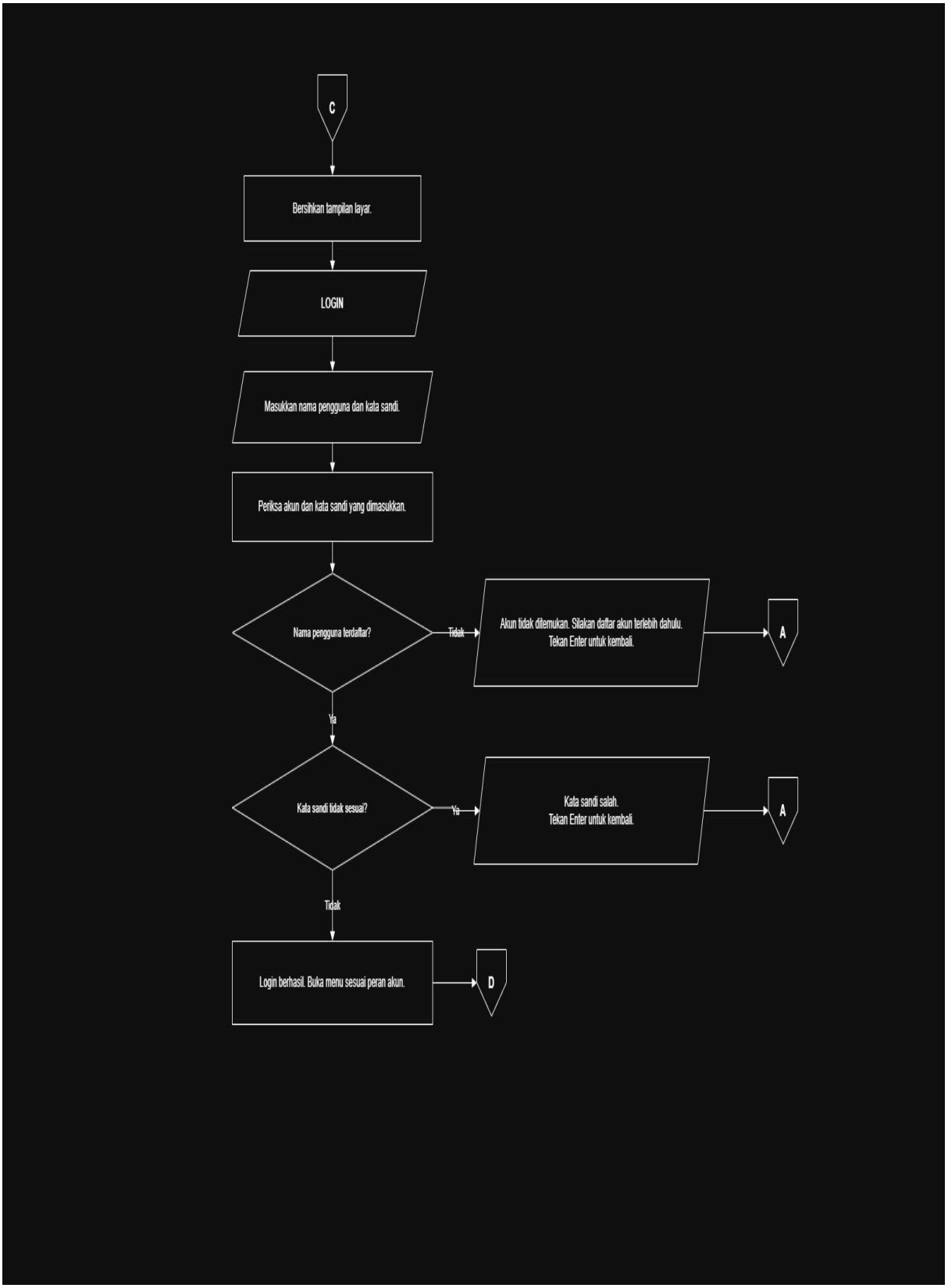
1. Flowchart



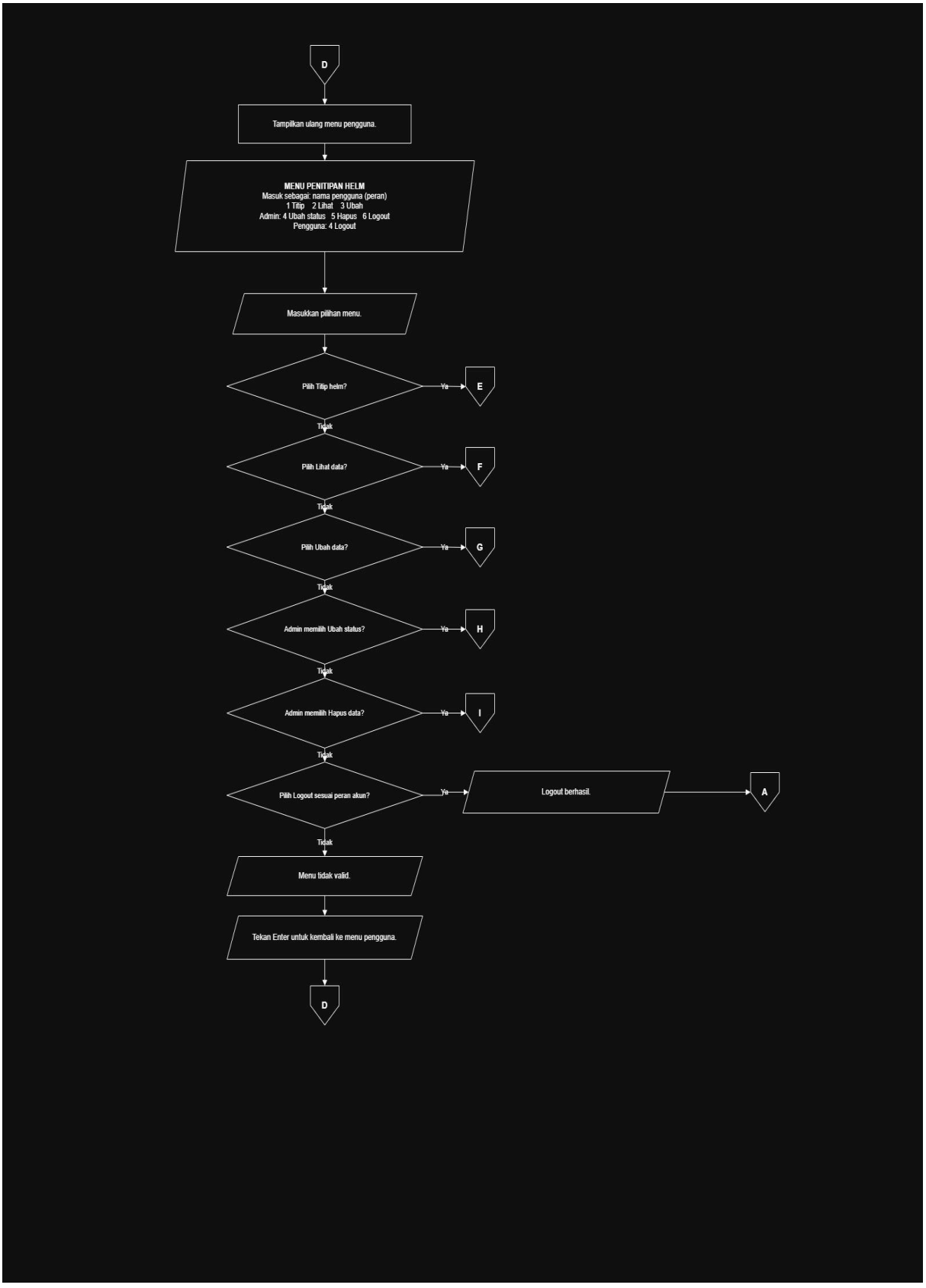
Gambar 1.1 Flowchart Halaman 1



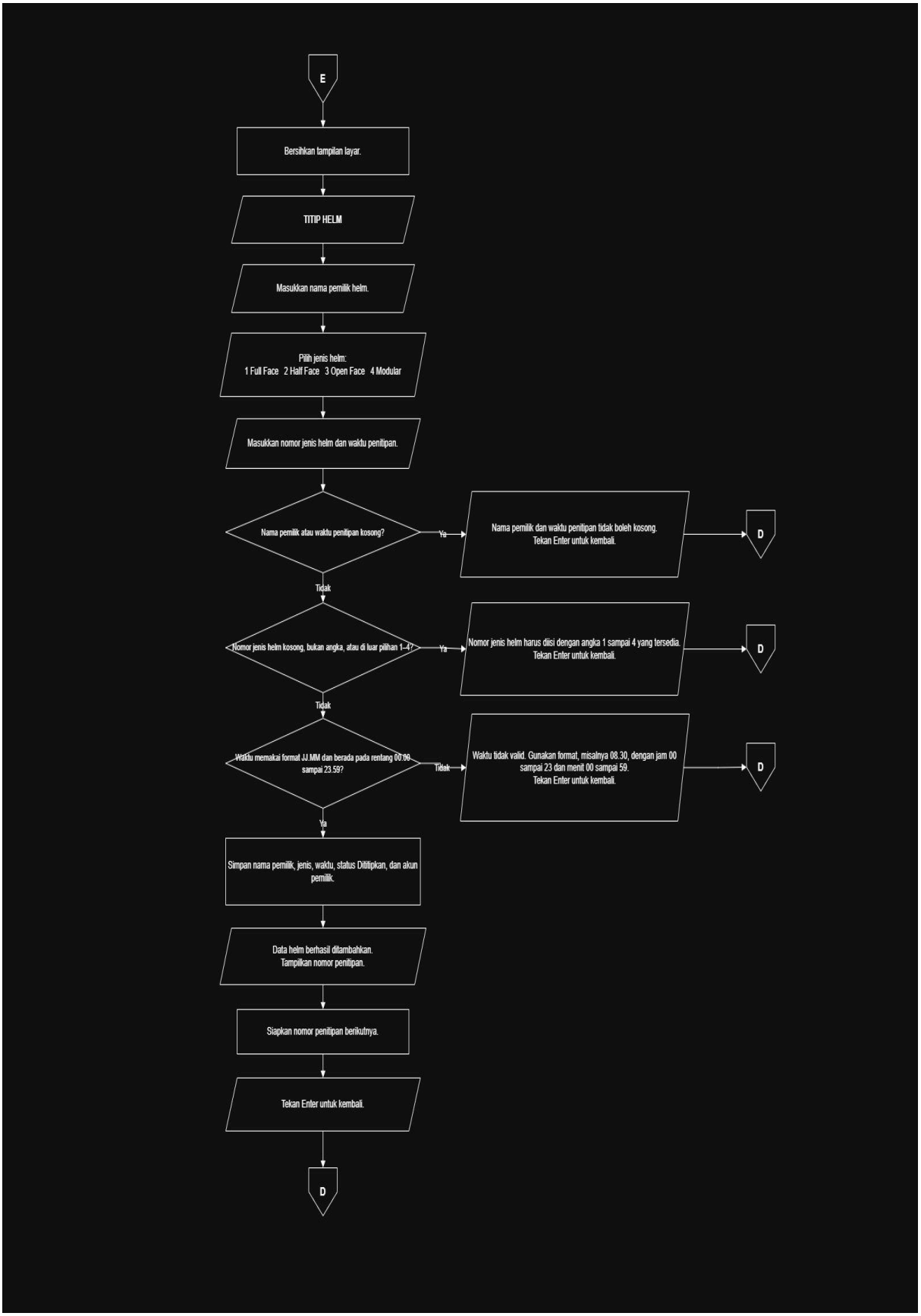
Gambar 1.2 Flowchart Halaman 2



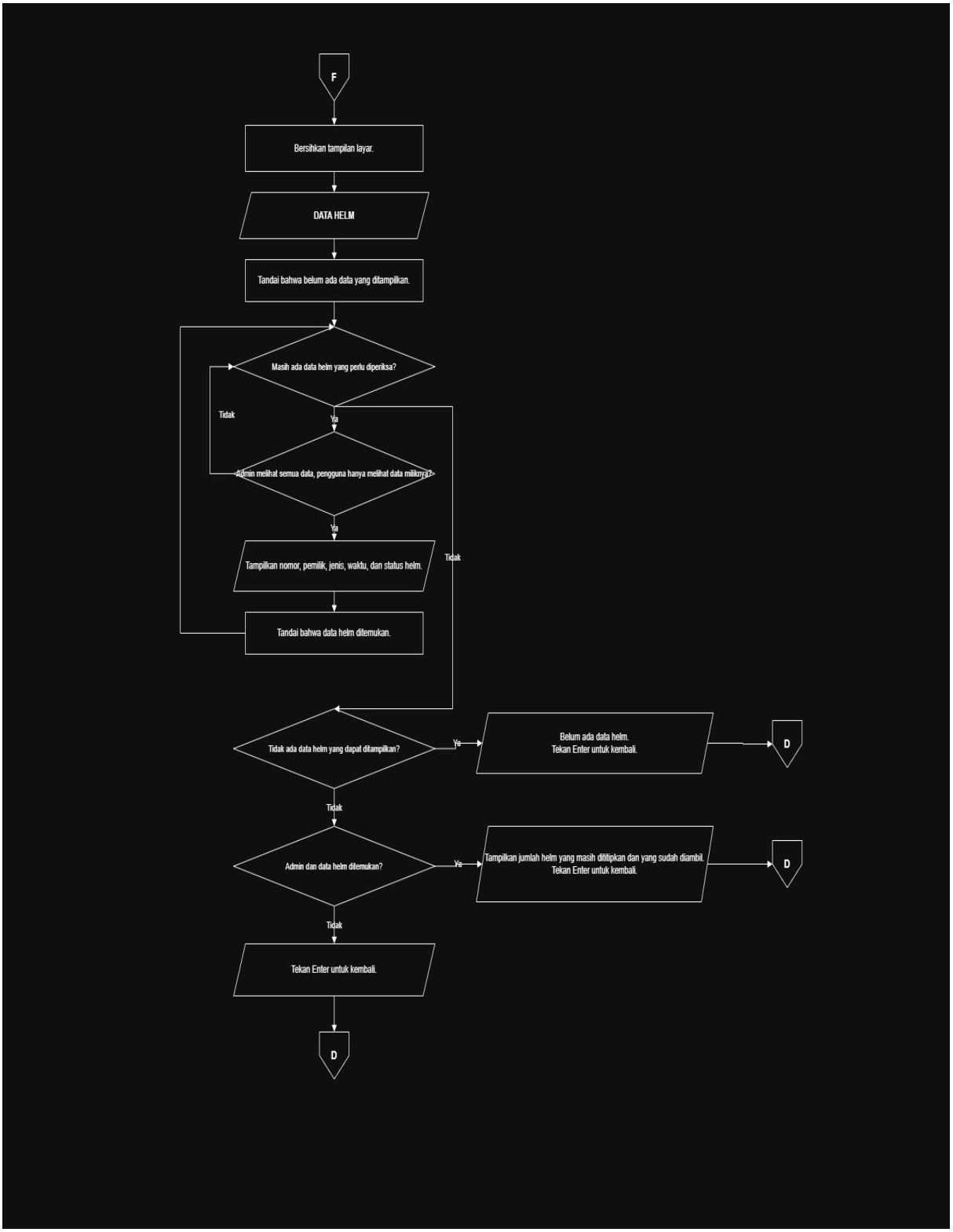
Gambar 1.3 Flowchart Halaman 3



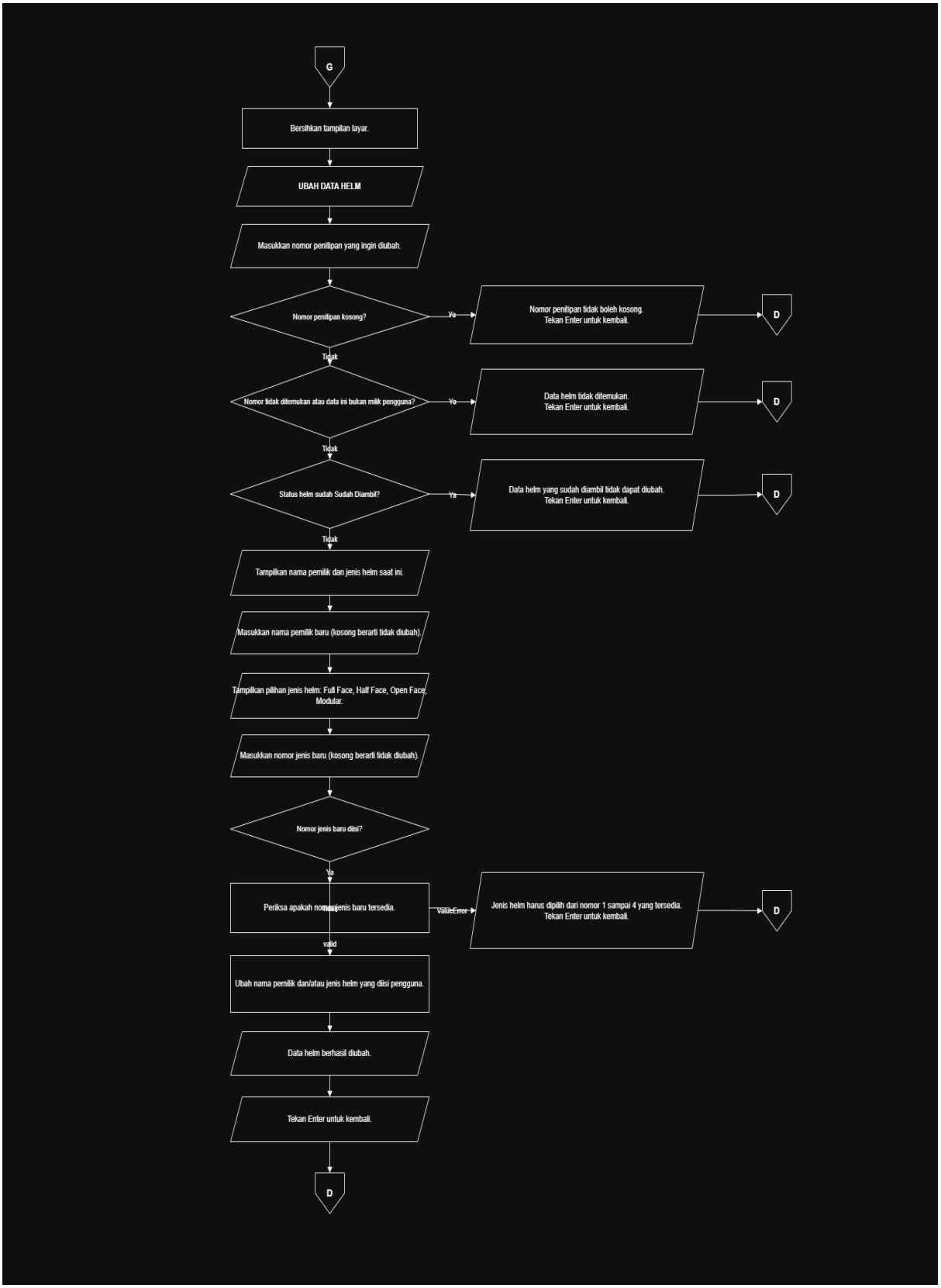
Gambar 1.4 Flowchart Halaman 4



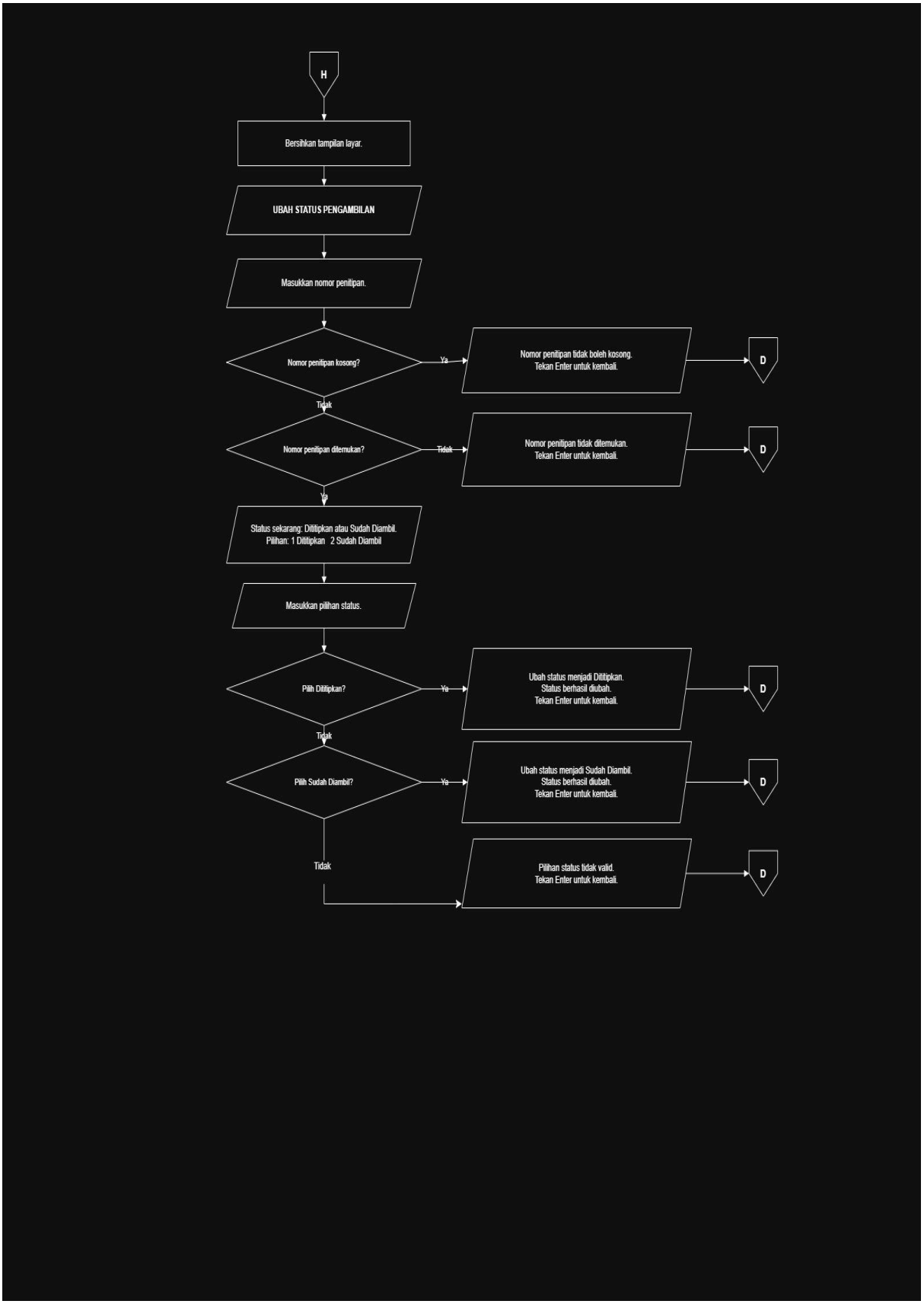
Gambar 1.5 Flowchart Halaman 5



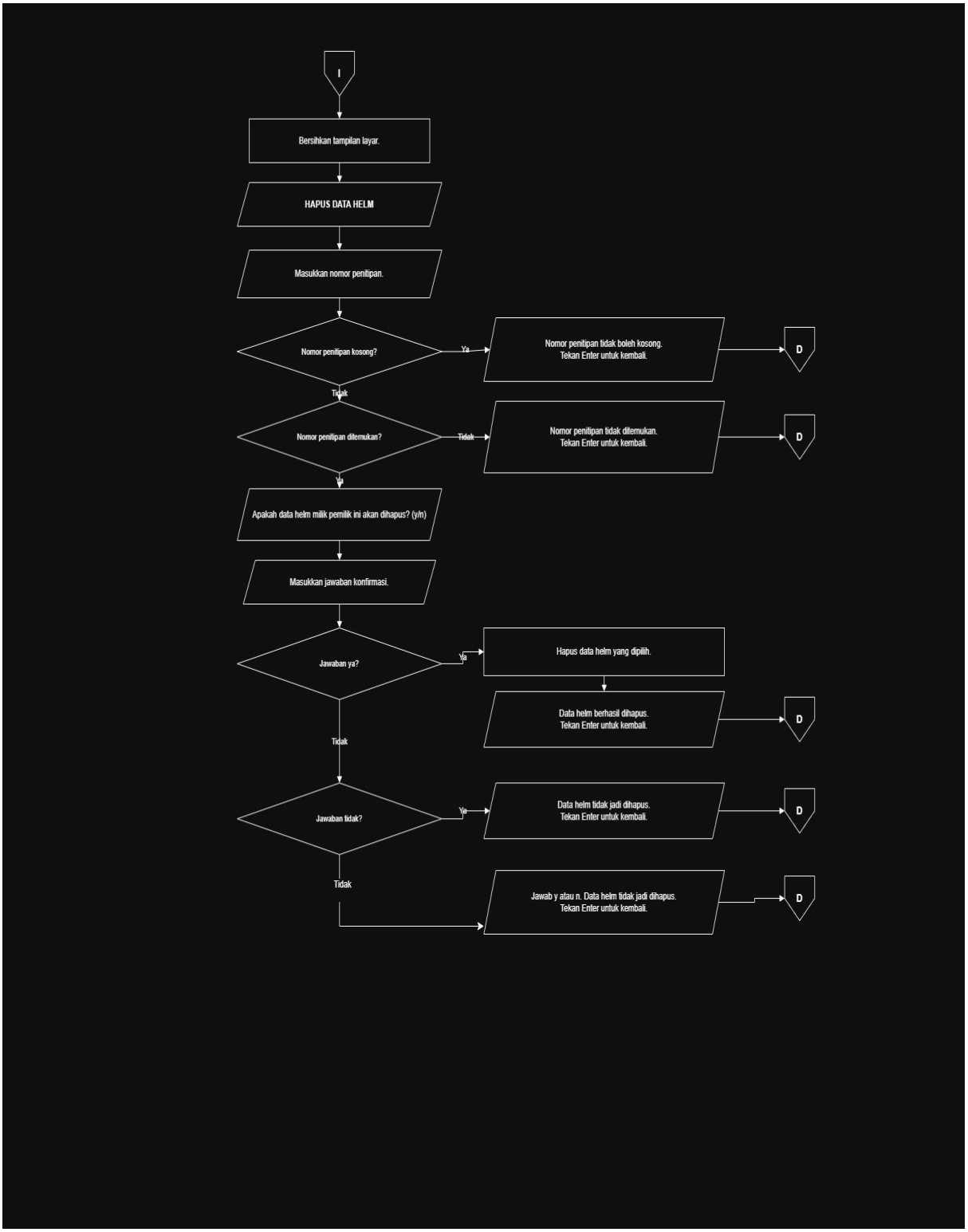
Gambar 1.6 Flowchart Halaman 6



Gambar 1.7 Flowchart Halaman 7



Gambar 1.8 Flowchart Halaman 8



Gambar 1.9 Flowchart Halaman 9

2. Deskripsi Singkat Program

Program ini merupakan sistem manajemen titip helm berbasis terminal yang dirancang secara terstruktur menggunakan bahasa Python. Pada awal eksekusi, program menyediakan menu utama yang menampilkan tiga opsi pilihan, yaitu Login, Daftar Akun, dan Keluar. Melalui fitur Daftar Akun, pengguna baru dapat mendaftarkan diri dengan memasukkan username dan password, di mana program menggunakan blok try-except untuk memastikan bahwa data masukan tidak kosong serta username belum pernah terdaftar sebelumnya di dalam kamus akun. Sistem secara default sudah menyediakan satu akun bawaan dengan peran admin, sementara akun baru yang didaftarkan secara mandiri akan otomatis mendapatkan peran sebagai user.

Setelah proses login berhasil, pengguna akan diarahkan ke menu utama atau menu_pengguna yang tampilannya disesuaikan secara dinamis berdasarkan hak akses atau role pengguna yang sedang aktif. Semua pengguna, baik admin maupun user, dapat mengakses fitur untuk menitipkan helm baru yang mencakup input nama pemilik, pemilihan jenis helm dari daftar tuple daftar_jenis, serta validasi format waktu ketat melalui fungsi cek_waktu, melihat daftar data helm, dan mengubah data helm. Khusus untuk pengguna dengan peran admin, tersedia menu tambahan yang memiliki hak istimewa, yaitu mengubah status pengambilan helm, menghapus data helm berdasarkan nomor penitipan, serta melihat ringkasan total keseluruhan helm berdasarkan statusnya menggunakan fungsi rekursif hitung_helm.

Dari segi teknis penyusunan kode, program ini menerapkan pemisahan tugas yang jelas antara fungsi dan prosedur serta memanfaatkan penanganan eksepsi menggunakan try-except untuk ValueError dan KeyError, raise ValueError, serta blok else dan finally. Struktur ini memastikan bahwa setiap kesalahan input yang dilakukan oleh pengguna, seperti salah format waktu, pemilihan menu yang tidak valid, atau pencarian nomor penitipan yang tidak ditemukan, dapat ditangani dengan baik oleh program tanpa menyebabkan penghentian paksa atau crash, sehingga memberikan pengalaman interaksi terminal yang aman dan stabil.

3. Source Code

```
import os

akun = {"admin": {"password": "admin123", "role": "admin"}}
helm = {}
nomor_berikutnya = 1
daftar_jenis = ("Full Face", "Half Face", "Open Face", "Modular")
angka = "0123456789"



def cek_waktu(waktu):
    waktu_valid = False
    if len(waktu) == 5 and waktu[2] == ".":
        if (waktu[0] in angka and waktu[1] in angka
                and waktu[3] in angka and waktu[4] in angka):
            if waktu[0:2] < "24" and waktu[3:5] < "60":
                waktu_valid = True
    return waktu_valid


def cari_jenis(pilih_jenis):
    jenis_dipilih = ""
    no = 1
    for jenis in daftar_jenis:
        if pilih_jenis == str(no):
            jenis_dipilih = jenis
        no += 1
    return jenis_dipilih



def menu_awal():
    os.system("cls")
    print("\n=== SISTEM TITIP HELM ===")
    print("1. Login")
    print("2. Daftar akun")
    print("3. Keluar")
    pilihan_awal = input("Masukkan menu yang dipilih: ")
    return pilihan_awal


def input_nomor():
    nomor_cari = input("Masukkan nomor penitipan: ")
    return nomor_cari
```

```python
def hitung_helm(daftar_nomor, indeks, status):
    if indeks == len(daftar_nomor):
        return 0
    if helm[daftar_nomor[indeks]]["status"] == status:
        return 1 + hitung_helm(daftar_nomor, indeks + 1, status)
    return hitung_helm(daftar_nomor, indeks + 1, status)



def tampilkan_jenis():
    no = 1
    for jenis in daftar_jenis:
        print(str(no) + ". " + jenis)
        no += 1


def daftar_akun():
    os.system("cls")
    print("\n=== DAFTAR AKUN ===")
    username_baru = input("Masukkan username baru: ")
    password_baru = input("Masukkan password baru: ")

    try:
        if username_baru == "" or password_baru == "":
            raise ValueError("Username dan password tidak boleh kosong.")
        elif username_baru in akun:
            raise ValueError("Username sudah digunakan. Silakan pakai
username lain.")
    except ValueError as e:
        print(e)
    else:
        akun[username_baru] = {"password": password_baru, "role": "user"}
        print("Akun berhasil ditambahkan. Silakan login.")
    input("Tekan Enter untuk kembali...")


def login():
    os.system("cls")
    print("\n=== LOGIN ===")
    username = input("Username: ")
    password = input("Password: ")

    try:
        if akun[username]["password"] != password:
            raise ValueError("Password salah.")
    except KeyError:
        print("Akun tidak ditemukan. Silakan daftar akun terlebih dahulu.")
```

```
        input("Tekan Enter untuk kembali...")
    except ValueError as e:
        print(e)
        input("Tekan Enter untuk kembali...")
    else:
        menu_pengguna(username, akun[username]["role"])


def titip_helm(username):
    global nomor_berikutnya
    os.system("cls")
    print("\n=== TITIP HELM ===")
    nama_pemilik = input("Masukkan nama pemilik helm: ")
    print("Pilih jenis helm:")
    tampilkan_jenis()
    pilih_jenis = input("Masukkan nomor jenis helm: ")
    waktu = input("Masukkan waktu penitipan (contoh 08.30): ")
    jenis_dipilih = cari_jenis(pilih_jenis)

    try:
        if nama_pemilik == "" or waktu == "":
            raise ValueError("Nama pemilik dan waktu penitipan tidak boleh
kosong.")
        elif jenis_dipilih == "":
            raise ValueError("Nomor jenis helm tidak tersedia.")
        elif cek_waktu(waktu) == False:
            raise ValueError("Format waktu tidak valid. Gunakan format
08.30.")
    except ValueError as e:
        print(e)
    else:
        helm[str(nomor_berikutnya)] = {
            "pemilik": nama_pemilik,
            "jenis": jenis_dipilih,
            "waktu": waktu,
            "status": "Dititipkan",
            "akun": username
        }
        print("Data helm berhasil ditambahkan. Nomor penitipan:",
nomor_berikutnya)
        nomor_berikutnya += 1
    finally:
        input("Tekan Enter untuk kembali...")


def lihat_data(role, username):
    os.system("cls")
    print("\n=== DATA HELM ===")
```

```
    ada_data = False

    for nomor, data in helm.items():
        if role == "admin" or data["akun"] == username:
            print("\nNomor   :", nomor)
            print("Pemilik :", data["pemilik"])
            print("Jenis   :", data["jenis"])
            print("Waktu   :", data["waktu"])
            print("Status  :", data["status"])
            ada_data = True

    if ada_data == False:
        print("Belum ada data helm.")
    elif role == "admin":
        daftar_nomor = list(helm.keys())
        print("\nTotal dititipkan    :", hitung_helm(daftar_nomor, 0,
"Dititipkan"))
        print("Total sudah diambil :", hitung_helm(daftar_nomor, 0, "Sudah
Diambil"))
    input("Tekan Enter untuk kembali...")


def ubah_data(role, username):
    os.system("cls")
    print("\n=== UBAH DATA HELM ===")
    nomor_cari = input_nomor()

    try:
        data_ketemu = helm[nomor_cari]
        if role != "admin" and data_ketemu["akun"] != username:
            raise ValueError("Data helm tidak ditemukan.")
        elif data_ketemu["status"] == "Sudah Diambil":
            raise ValueError("Data helm yang sudah diambil tidak dapat
diubah.")

        print("Nama pemilik saat ini:", data_ketemu["pemilik"])
        print("Jenis helm saat ini  :", data_ketemu["jenis"])
        nama_baru = input("Masukkan nama pemilik baru (kosongkan jika
tetap): ")
        print("Pilih jenis helm baru:")
        tampilkan_jenis()
        jenis_baru = input("Masukkan nomor jenis baru (kosongkan jika
tetap): ")
        jenis_dipilih = cari_jenis(jenis_baru)

        if jenis_baru != "" and jenis_dipilih == "":
            raise ValueError("Nomor jenis helm tidak tersedia.")
    except KeyError:
```

```
        print("Data helm tidak ditemukan.")
    except ValueError as e:
        print(e)
    else:
        if nama_baru != "":
            data_ketemu["pemilik"] = nama_baru
        if jenis_dipilih != "":
            data_ketemu["jenis"] = jenis_dipilih
        print("Data helm berhasil diubah.")
    input("Tekan Enter untuk kembali...")


def ubah_status():
    os.system("cls")
    print("\n=== UBAH STATUS PENGAMBILAN ===")
    nomor_cari = input_nomor()

    try:
        print("Status sekarang:", helm[nomor_cari]["status"])
        print("1. Dititipkan")
        print("2. Sudah diambil")
        pilih_status = input("Masukkan status baru: ")

        if pilih_status == "1":
            helm[nomor_cari]["status"] = "Dititipkan"
        elif pilih_status == "2":
            helm[nomor_cari]["status"] = "Sudah Diambil"
        else:
            raise ValueError("Pilihan status tidak valid.")
    except KeyError:
        print("Nomor penitipan tidak ditemukan.")
    except ValueError as e:
        print(e)
    else:
        print("Status berhasil diubah.")
    input("Tekan Enter untuk kembali...")


def hapus_data():
    os.system("cls")
    print("\n=== HAPUS DATA HELM ===")
    nomor_cari = input_nomor()

    try:
        konfirmasi = input(
            "Hapus data helm milik " + helm[nomor_cari]["pemilik"] + "?
(y/n): "
        )
```

```
        if konfirmasi == "y" or konfirmasi == "Y":
            del helm[nomor_cari]
            print("Data helm berhasil dihapus.")
        elif konfirmasi == "n" or konfirmasi == "N":
            print("Data helm tidak jadi dihapus.")
        else:
            print("Jawab y atau n. Data helm tidak jadi dihapus.")
    except KeyError:
        print("Nomor penitipan tidak ditemukan.")
    input("Tekan Enter untuk kembali...")


def menu_pengguna(username, role):
    while True:
        os.system("cls")
        print("\n=== MENU PENITIPAN HELM ===")
        print("Login sebagai:", username, "(" + role + ")")
        print("1. Titip helm")
        print("2. Lihat data helm")
        print("3. Ubah data helm")

        if role == "admin":
            print("4. Ubah status pengambilan")
            print("5. Hapus data helm")
            print("6. Logout")
            kode_logout = "6"
        else:
            print("4. Logout")
            kode_logout = "4"

        pilihan = input("Masukkan menu yang dipilih: ")

        if pilihan == "1":
            titip_helm(username)
        elif pilihan == "2":
            lihat_data(role, username)
        elif pilihan == "3":
            ubah_data(role, username)
        elif pilihan == "4" and role == "admin":
            ubah_status()
        elif pilihan == "5" and role == "admin":
            hapus_data()
        elif pilihan == kode_logout:
            print("Logout berhasil.")
            break
        else:
            print("Menu tidak valid.")
```

```
            input("Tekan Enter untuk kembali...")



while True:
    pilihan_awal = menu_awal()

    if pilihan_awal == "1":
        login()
    elif pilihan_awal == "2":
        daftar_akun()
    elif pilihan_awal == "3":
        print("Program selesai. Terima kasih sudah menggunakan Sistem Titip
Helm.")
        break
    else:
        print("Menu tidak valid.")
        input("Tekan Enter untuk kembali...")
```

A. Inisialisasi Data dan Variabel Global

Program diawali dengan import os untuk membersihkan layar lewat os.system("cls"). Data utama disimpan sebagai variabel global di luar fungsi akun (dictionary akun, awalnya berisi satu admin), helm (dictionary kosong yang nanti diisi data penitipan, dengan nomor penitipan sebagai key), dan nomor_berikutnya (penghitung nomor penitipan yang mulai dari 1). Tuple daftar_jenis menjadi pilihan jenis helm, sedangkan string angka dipakai untuk memeriksa apakah karakter waktu berupa digit.

Source Code:

```
import os

akun = {"admin": {"password": "admin123", "role": "admin"}}
helm = {}
nomor_berikutnya = 1
daftar_jenis = ("Full Face", "Half Face", "Open Face", "Modular")
angka = "0123456789"
```

B. Fungsi Pendukung (Validasi Waktu, Pilihan Jenis, dan Tampilan)

cek_waktu() memvalidasi format waktu HH.MM: panjangnya harus 5 karakter, karakter ketiga berupa titik, sisanya digit, jam kurang dari 24 dan menit kurang dari 60. cari_jenis() mengubah nomor pilihan menjadi nama jenis helm dengan mengulang tuple daftar_jenis, dan mengembalikan string kosong jika nomor tidak tersedia. tampilkan_jenis() mencetak daftar jenis helm bernomor, dan input_nomor() meminta nomor penitipan dari pengguna.

Source Code:

```
def cek_waktu(waktu):
    waktu_valid = False
    if len(waktu) == 5 and waktu[2] == ".":
        if (waktu[0] in angka and waktu[1] in angka
                and waktu[3] in angka and waktu[4] in angka):
            if waktu[0:2] < "24" and waktu[3:5] < "60":
                waktu_valid = True
    return waktu_valid


def cari_jenis(pilih_jenis):
    jenis_dipilih = ""
    no = 1
    for jenis in daftar_jenis:
        if pilih_jenis == str(no):
            jenis_dipilih = jenis
        no += 1
    return jenis_dipilih


def input_nomor():
    nomor_cari = input("Masukkan nomor penitipan: ")
    return nomor_cari


def tampilkan_jenis():
    no = 1
    for jenis in daftar_jenis:
        print(str(no) + ". " + jenis)
        no += 1
```

C. Fungsi Rekursif Hitung Helm

hitung_helm() menghitung jumlah helm berdasarkan status secara rekursif. Fungsi menerima daftar nomor penitipan, indeks yang sedang diperiksa, dan status yang dicari. Base case-nya adalah ketika indeks sama dengan panjang daftar, lalu fungsi mengembalikan 0. Jika status helm pada indeks tersebut cocok, fungsi mengembalikan 1 ditambah hasil pemanggilan berikutnya, jika tidak, langsung lanjut ke indeks berikutnya. Fungsi ini dipakai admin untuk menampilkan total helm yang dititipkan dan yang sudah diambil.

Source Code:

```
def hitung_helm(daftar_nomor, indeks, status):
    if indeks == len(daftar_nomor):
        return 0
    if helm[daftar_nomor[indeks]]["status"] == status:
        return 1 + hitung_helm(daftar_nomor, indeks + 1, status)
    return hitung_helm(daftar_nomor, indeks + 1, status)
```

D. Menu Awal

menu_awal() membersihkan layar, menampilkan tiga pilihan (Login, Daftar akun, Keluar), lalu mengembalikan pilihan yang diketik pengguna ke program utama.

Source Code:

```
def menu_awal():
    os.system("cls")
    print("\n=== SISTEM TITIP HELM ===")
    print("1. Login")
    print("2. Daftar akun")
    print("3. Keluar")
    pilihan_awal = input("Masukkan menu yang dipilih: ")
    return pilihan_awal
```

E. Daftar Akun

Fitur ini membuat akun baru dengan role user. Pengguna mengisi username dan password, lalu blok try memeriksa dua kondisi: input tidak boleh kosong dan username belum boleh dipakai. Jika ada yang gagal, raise ValueError dilempar dan pesannya ditampilkan oleh except. Jika lolos, blok else menyimpan akun ke dictionary akun.

Source Code:

```
def daftar_akun():
    os.system("cls")
    print("\n=== DAFTAR AKUN ===")
    username_baru = input("Masukkan username baru: ")
    password_baru = input("Masukkan password baru: ")

    try:
        if username_baru == "" or password_baru == "":
            raise ValueError("Username dan password tidak boleh kosong.")
        elif username_baru in akun:
            raise ValueError("Username sudah digunakan. Silakan pakai
username lain.")
    except ValueError as e:
        print(e)
    else:
        akun[username_baru] = {"password": password_baru, "role": "user"}
        print("Akun berhasil ditambahkan. Silakan login.")
    input("Tekan Enter untuk kembali...")
```

F. Login

Fitur login memeriksa username dan password. Jika username tidak ada di dictionary, Python melempar KeyError yang ditangkap dengan pesan akun tidak ditemukan. Jika password tidak cocok, program melempar ValueError "Password salah". Jika berhasil, blok else memanggil menu_pengguna() dengan membawa username dan role.

Source Code:

```
def login():
    os.system("cls")
    print("\n=== LOGIN ===")
    username = input("Username: ")
    password = input("Password: ")

    try:
        if akun[username]["password"] != password:
            raise ValueError("Password salah.")
    except KeyError:
        print("Akun tidak ditemukan. Silakan daftar akun terlebih dahulu.")
        input("Tekan Enter untuk kembali...")
    except ValueError as e:
        print(e)
        input("Tekan Enter untuk kembali...")
    else:
        menu_pengguna(username, akun[username]["role"])
```

G. Titip Helm

Fitur ini menambahkan data penitipan baru. Pengguna mengisi nama pemilik, jenis helm, dan waktu penitipan. Validasi dilakukan di blok try: nama dan waktu tidak boleh kosong, nomor jenis harus tersedia, dan format waktu harus valid lewat cek_waktu(). Jika lolos, data disimpan ke dictionary helm dengan key nomor_berikutnya (berupa string), status awal "Dititipkan", dan field akun yang mencatat siapa pemilik datanya. Setelah itu nomor_berikutnya ditambah 1. Blok finally memastikan pesan "Tekan Enter" selalu muncul.

Source Code:

```
def titip_helm(username):
    global nomor_berikutnya
    os.system("cls")
    print("\n=== TITIP HELM ===")
    nama_pemilik = input("Masukkan nama pemilik helm: ")
```

```
    print("Pilih jenis helm:")
    tampilkan_jenis()
    pilih_jenis = input("Masukkan nomor jenis helm: ")
    waktu = input("Masukkan waktu penitipan (contoh 08.30): ")
    jenis_dipilih = cari_jenis(pilih_jenis)

    try:
        if nama_pemilik == "" or waktu == "":
            raise ValueError("Nama pemilik dan waktu penitipan tidak boleh
kosong.")
        elif jenis_dipilih == "":
            raise ValueError("Nomor jenis helm tidak tersedia.")
        elif cek_waktu(waktu) == False:
            raise ValueError("Format waktu tidak valid. Gunakan format
08.30.")
    except ValueError as e:
        print(e)
    else:
        helm[str(nomor_berikutnya)] = {
            "pemilik": nama_pemilik,
            "jenis": jenis_dipilih,
            "waktu": waktu,
            "status": "Dititipkan",
            "akun": username
        }
        print("Data helm berhasil ditambahkan. Nomor penitipan:",
nomor_berikutnya)
        nomor_berikutnya += 1
    finally:
        input("Tekan Enter untuk kembali...")
```

H. Lihat Data Helm

Fitur ini menampilkan data helm. Admin melihat semua data, sedangkan user hanya melihat data miliknya sendiri (dicocokkan lewat field akun). Jika tidak ada data yang tampil, muncul pesan “Belum ada data helm”. Khusus admin, di bagian bawah ditampilkan total helm yang dititipkan dan yang sudah diambil menggunakan fungsi rekursif hitung_helm().

Source Code:

```
def lihat_data(role, username):
    os.system("cls")
    print("\n=== DATA HELM ===")
    ada_data = False
```

```
    for nomor, data in helm.items():
        if role == "admin" or data["akun"] == username:
            print("\nNomor   :", nomor)
            print("Pemilik :", data["pemilik"])
            print("Jenis   :", data["jenis"])
            print("Waktu   :", data["waktu"])
            print("Status  :", data["status"])
            ada_data = True

    if ada_data == False:
        print("Belum ada data helm.")
    elif role == "admin":
        daftar_nomor = list(helm.keys())
        print("\nTotal dititipkan    :", hitung_helm(daftar_nomor, 0,
"Dititipkan"))
        print("Total sudah diambil :", hitung_helm(daftar_nomor, 0, "Sudah
Diambil"))
    input("Tekan Enter untuk kembali...")
```

I. Ubah Data Helm

Fitur ini mengubah nama pemilik dan jenis helm. Nomor yang tidak ada memicu KeyError. User biasa hanya boleh mengubah data miliknya, jika bukan, pesan yang muncul sama dengan data tidak ditemukan. Data yang sudah berstatus “Sudah Diambil” tidak dapat diubah. Kolom yang dikosongkan berarti nilainya tetap, dan nomor jenis yang tidak valid ditolak dengan ValueError. Perubahan baru diterapkan di blok else setelah semua validasi lolos.

Source Code:

```
def ubah_data(role, username):
    os.system("cls")
    print("\n=== UBAH DATA HELM ===")
    nomor_cari = input_nomor()

    try:
        data_ketemu = helm[nomor_cari]
        if role != "admin" and data_ketemu["akun"] != username:
            raise ValueError("Data helm tidak ditemukan.")
        elif data_ketemu["status"] == "Sudah Diambil":
            raise ValueError("Data helm yang sudah diambil tidak dapat
diubah.")

        print("Nama pemilik saat ini:", data_ketemu["pemilik"])
        print("Jenis helm saat ini  :", data_ketemu["jenis"])
```

```
        nama_baru = input("Masukkan nama pemilik baru (kosongkan jika
tetap): ")
        print("Pilih jenis helm baru:")
        tampilkan_jenis()
        jenis_baru = input("Masukkan nomor jenis baru (kosongkan jika
tetap): ")
        jenis_dipilih = cari_jenis(jenis_baru)

        if jenis_baru != "" and jenis_dipilih == "":
            raise ValueError("Nomor jenis helm tidak tersedia.")
    except KeyError:
        print("Data helm tidak ditemukan.")
    except ValueError as e:
        print(e)
    else:
        if nama_baru != "":
            data_ketemu["pemilik"] = nama_baru
        if jenis_dipilih != "":
            data_ketemu["jenis"] = jenis_dipilih
        print("Data helm berhasil diubah.")
    input("Tekan Enter untuk kembali...")
```

J. Ubah Status Pengambilan (Khusus Admin)

Fitur ini hanya muncul di menu admin. Admin memasukkan nomor penitipan, melihat status saat ini, lalu memilih status baru: “Dititipkan” atau “Sudah Diambil”. Nomor yang tidak ada ditangkap oleh KeyError, dan pilihan status di luar 1 atau 2 memicu ValueError.

Source Code:

```
def ubah_status():
    os.system("cls")
    print("\n=== UBAH STATUS PENGAMBILAN ===")
    nomor_cari = input_nomor()

    try:
        print("Status sekarang:", helm[nomor_cari]["status"])
        print("1. Dititipkan")
        print("2. Sudah diambil")
        pilih_status = input("Masukkan status baru: ")

        if pilih_status == "1":
            helm[nomor_cari]["status"] = "Dititipkan"
        elif pilih_status == "2":
            helm[nomor_cari]["status"] = "Sudah Diambil"
        else:
```

```
            raise ValueError("Pilihan status tidak valid.")
    except KeyError:
        print("Nomor penitipan tidak ditemukan.")
    except ValueError as e:
        print(e)
    else:
        print("Status berhasil diubah.")
    input("Tekan Enter untuk kembali...")
```

K. Hapus Data Helm (Khusus Admin)

Admin memasukkan nomor penitipan, lalu program meminta konfirmasi (y/n) dengan menampilkan nama pemilik. Jika y, data dihapus dengan del. Jika n atau jawaban lain, data tidak dihapus. Nomor yang tidak ditemukan ditangani oleh KeyError.

Source Code:

```
def hapus_data():
    os.system("cls")
    print("\n=== HAPUS DATA HELM ===")
    nomor_cari = input_nomor()

    try:
        konfirmasi = input(
            "Hapus data helm milik " + helm[nomor_cari]["pemilik"] + "?
(y/n): "
        )

        if konfirmasi == "y" or konfirmasi == "Y":
            del helm[nomor_cari]
            print("Data helm berhasil dihapus.")
        elif konfirmasi == "n" or konfirmasi == "N":
            print("Data helm tidak jadi dihapus.")
        else:
            print("Jawab y atau n. Data helm tidak jadi dihapus.")
    except KeyError:
        print("Nomor penitipan tidak ditemukan.")
    input("Tekan Enter untuk kembali...")
```

L. Menu Pengguna Berdasarkan Role

menu_pengguna() menampilkan menu setelah login dalam perulangan while True. Menu 1 sampai 3 (titip, lihat, ubah data) tersedia untuk semua role. Admin mendapat tambahan menu 4 (ubah status) dan 5 (hapus data), dengan logout di nomor 6, sedangkan user biasa punya logout di nomor 4. Variabel kode_logout menyimpan nomor logout sesuai role, sehingga satu fungsi dapat melayani kedua role.

Source Code:

```
def menu_pengguna(username, role):
    while True:
        os.system("cls")
        print("\n=== MENU PENITIPAN HELM ===")
        print("Login sebagai:", username, "(" + role + ")")
        print("1. Titip helm")
        print("2. Lihat data helm")
        print("3. Ubah data helm")

        if role == "admin":
            print("4. Ubah status pengambilan")
            print("5. Hapus data helm")
            print("6. Logout")
            kode_logout = "6"
        else:
            print("4. Logout")
            kode_logout = "4"

        pilihan = input("Masukkan menu yang dipilih: ")

        if pilihan == "1":
            titip_helm(username)
        elif pilihan == "2":
            lihat_data(role, username)
        elif pilihan == "3":
            ubah_data(role, username)
        elif pilihan == "4" and role == "admin":
            ubah_status()
        elif pilihan == "5" and role == "admin":
            hapus_data()
        elif pilihan == kode_logout:
            print("Logout berhasil.")
            break
        else:
            print("Menu tidak valid.")
            input("Tekan Enter untuk kembali...")
```

M. Program Utama

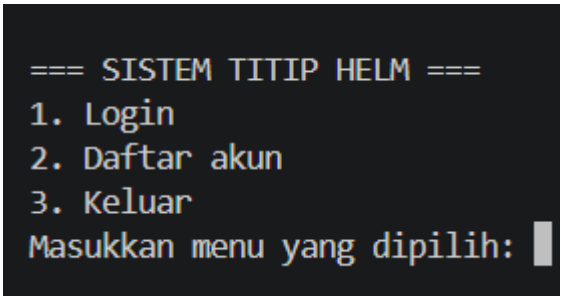
Bagian ini adalah perulangan utama program. Setiap putaran memanggil menu_awal() lalu menjalankan fungsi sesuai pilihan: login, daftar akun, atau keluar (menghentikan perulangan dengan break). Pilihan di luar menu menampilkan pesan tidak valid.

Source Code:

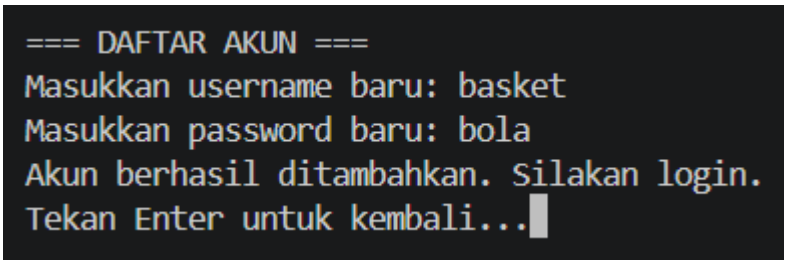
```
while True:
    pilihan_awal = menu_awal()

    if pilihan_awal == "1":
        login()
    elif pilihan_awal == "2":
        daftar_akun()
    elif pilihan_awal == "3":
        print("Program selesai. Terima kasih sudah menggunakan Sistem Titip
Helm.")
        break
    else:
        print("Menu tidak valid.")
        input("Tekan Enter untuk kembali...")
```

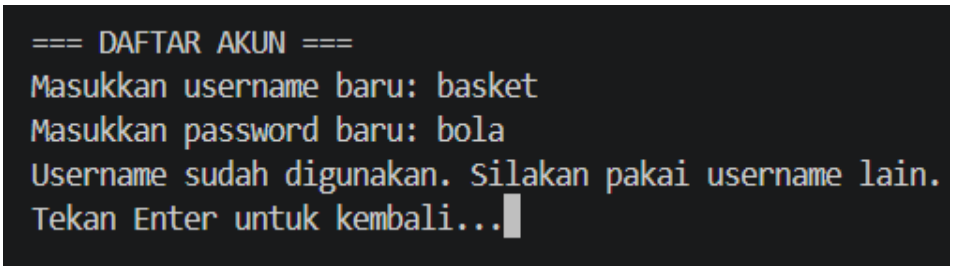
4. Hasil Output



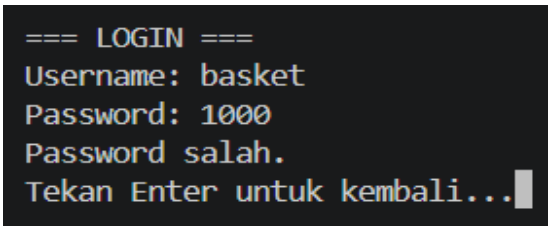
4.1 Menu awal (Login, Daftar akun, Keluar)



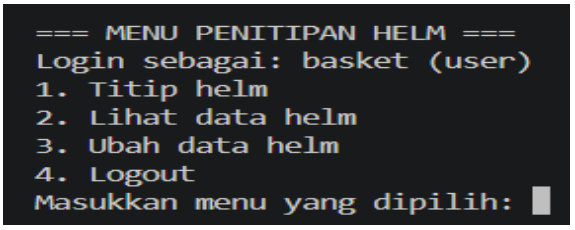
4.2 Daftar akun berhasil



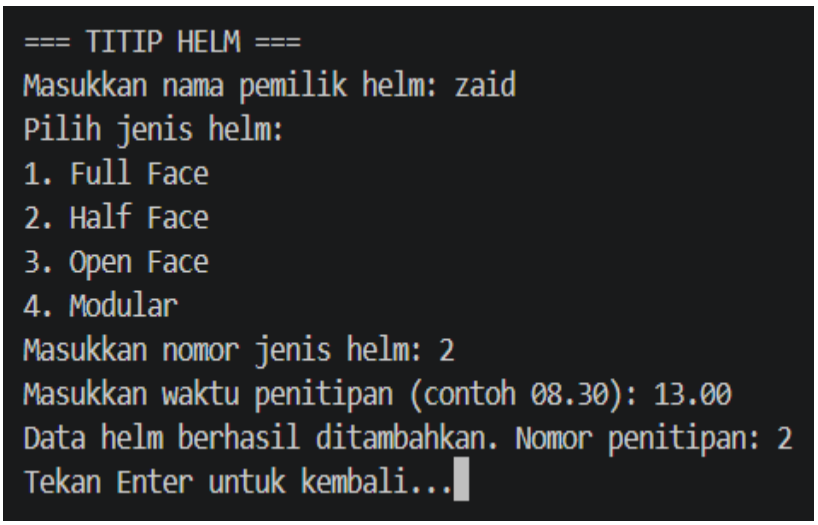
4.3 Gambar Tampilan Daftar Akun Gagal.



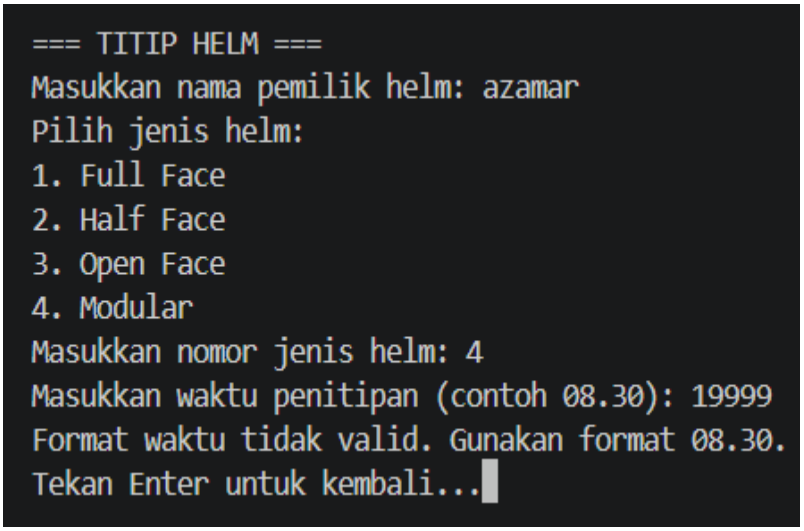
4.4 Gambar Tampilan Login Gagal.



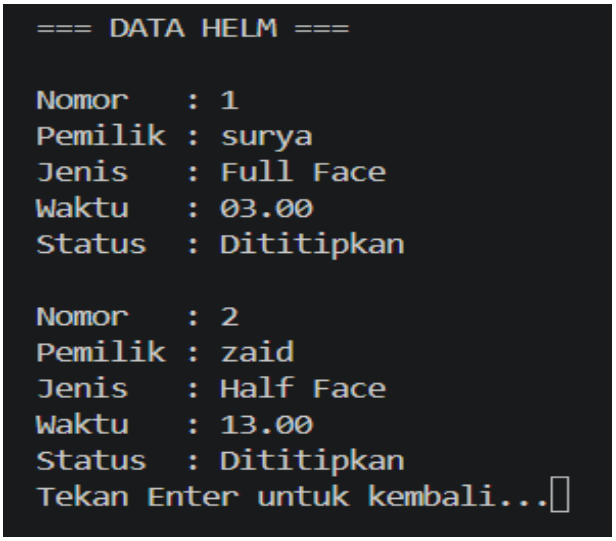
4.5 Gambar Tampilan Menu User.



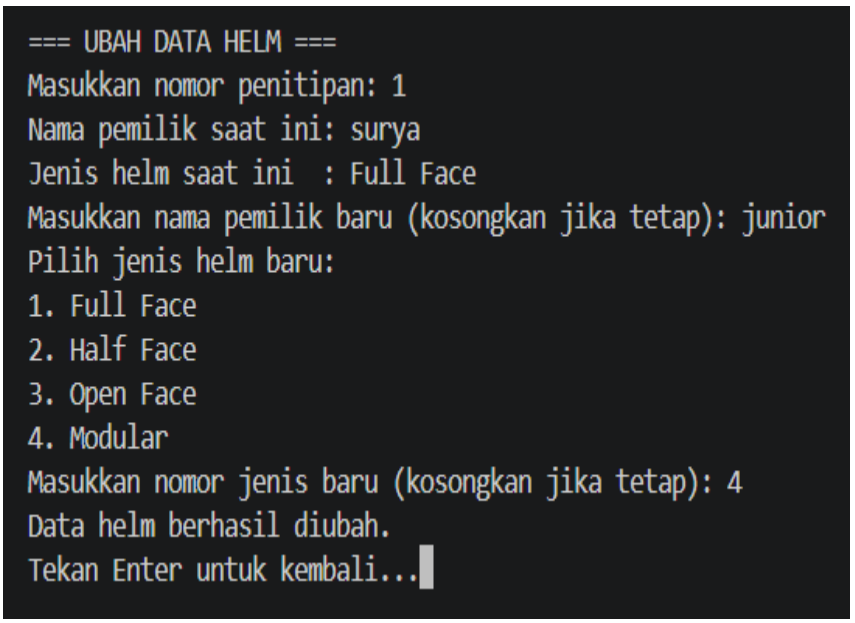
4.6 Gambar Tampilan Titip Helm Berhasil.



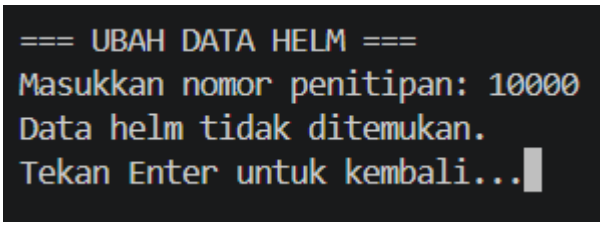
4.7 Gambar Tampilan Titip Helm Gagal.



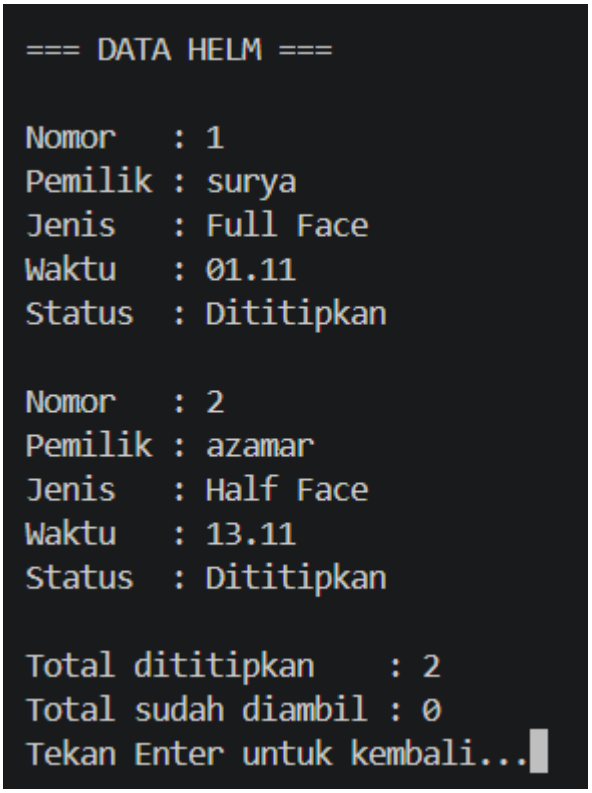
4.8 Gambar Tampilan Lihat Data Helm (User).



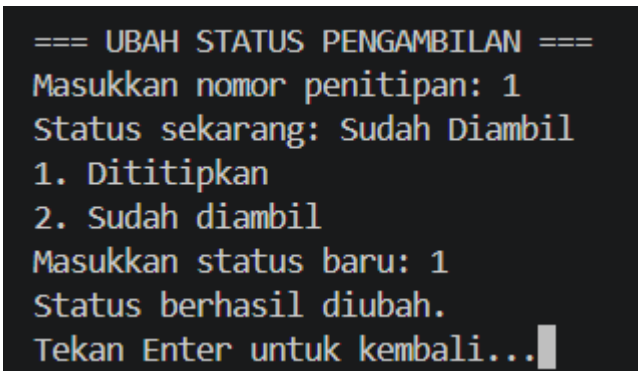
4.9 Gambar Tampilan Ubah Data Berhasil.



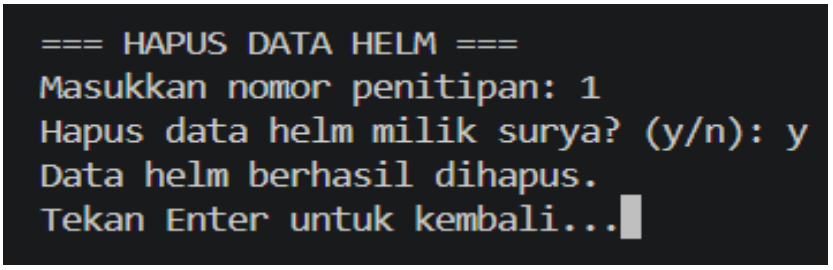
4.10 Gambar Tampilan Ubah Data Gagal.



4.13 Gambar Tampilan Lihat Data Helm (Admin).



4.14 Gambar Tampilan Ubah Status Pengambilan.

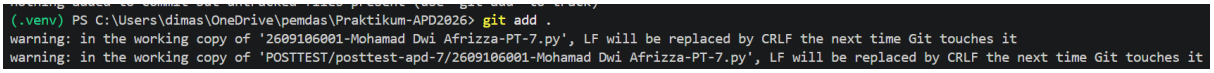


4.15 Gambar Tampilan Hapus Data.

5. Langkah-Langkah GIT

5.1 GIT ADD

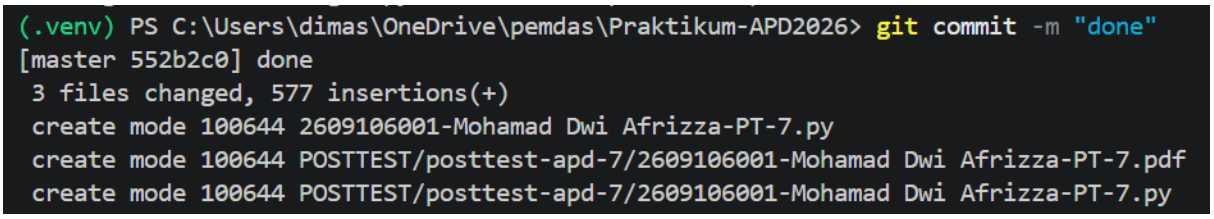
Perintah git add digunakan untuk memasukkan file yang akan disimpan ke dalam proses Git.



5.1 Gambar GIT ADD

5.2 GIT COMMIT

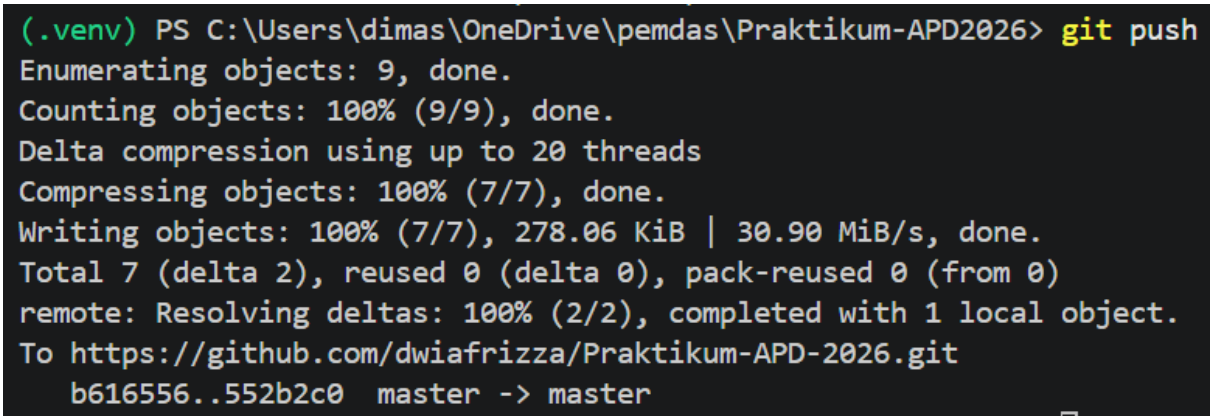
Perintah git commit digunakan untuk menyimpan perubahan file sebagai riwayat pada repository.



5.2 Gambar GIT COMMIT

5.3 GIT PUSH

Perintah git push digunakan untuk mengirim perubahan dari komputer ke repository GitHub.



5.3 Gambar GIT PUSH